

NAME:VAISHNAL MALI
DIV:D15A ROLL NO.:27
ADV-DEVOPS CASE-STUDY-7

Kubernetes Cluster Management with Terraform

- Concepts Used: Kubernetes, Terraform, AWS CloudShell.
- Problem Statement: "Use Terraform to provision a Kubernetes cluster on AWS. Then, use AWS CloudShell IDE to deploy a sample application on the cluster using kubectl."
- Tasks:
 - Write a Terraform script to create a Kubernetes cluster on AWS.
 - Use AWS CloudShell to configure kubectl for the newly created cluster.
 - Deploy a simple application (e.g., a Python Flask app) on the Kubernetes cluster and verify its deployment.

1.Introduction:

Case Study Overview

This case study focuses on the management of Kubernetes clusters using Terraform within the AWS cloud environment. Kubernetes, an open-source container orchestration platform, automates the deployment, scaling, and management of containerized applications. Terraform, an infrastructure as code tool, allows for the efficient provisioning and management of cloud resources. The integration of these technologies enables the creation of scalable, resilient, and manageable cloud-native applications.

Key Feature and Application

One of the key features highlighted in this case study is the use of Terraform to provision a Kubernetes cluster on AWS seamlessly. This feature allows developers and system administrators to define their infrastructure as code, facilitating version control, collaboration, and consistency across environments. By leveraging AWS services like Elastic Kubernetes Service (EKS), Terraform scripts can create an entire Kubernetes cluster, including necessary networking and security configurations, with minimal manual intervention.

The practical application of this case study lies in its ability to empower teams to deploy complex applications with ease. For instance, by using Kubernetes, organizations can ensure that their applications are resilient to failure, can scale dynamically based on demand, and are easier to manage. The deployment of a sample application, such as a Python Flask app, demonstrates the power of this integration and showcases how businesses can accelerate their application development and deployment processes.

Third-Year Project Integration

In relation to my third-year project, which involves building a comprehensive cloud-native application, this case study directly aligns with my objectives. By understanding the principles of Kubernetes and Terraform, I can apply these concepts to manage my application's infrastructure efficiently. The experience gained from this case study will inform the design and deployment of my project, ensuring that it is scalable and maintains high availability.

Concepts Used

This case study utilizes the following concepts:

- **Kubernetes:** For managing containerized applications and orchestrating their deployment.
- **Terraform:** For provisioning the Kubernetes cluster and automating infrastructure management.
- **AWS CloudShell:** For executing commands in a cloud environment without needing local setup, simplifying the deployment process.

Tasks

1. **Write a Terraform Script:** The first task is to create a Terraform script that provisions a Kubernetes cluster on AWS. This includes defining resources such as the EKS cluster, node groups, VPC settings, and security groups. By utilizing Terraform modules and AWS provider configurations, the script will automate the entire setup process, ensuring that all components are configured correctly.
2. **Use AWS CloudShell to Configure kubectl:** Once the Kubernetes cluster is provisioned, the next step is to configure kubectl, the command-line tool for interacting with Kubernetes. This will be done using AWS CloudShell, which provides a pre-configured environment for managing AWS resources. The

necessary credentials and context will be set up to allow seamless interaction with the newly created cluster.

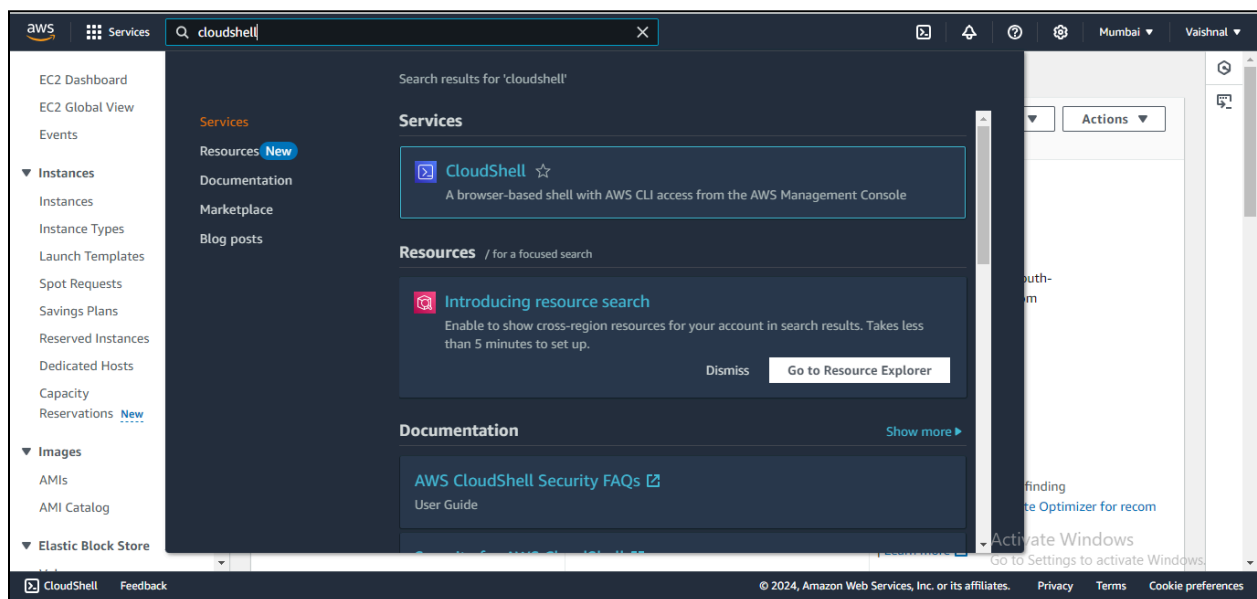
3. **Deploy a Simple Application:** Finally, a simple application, such as a Python Flask app, will be deployed onto the Kubernetes cluster. This will involve creating deployment and service YAML files to define the application's pods and expose it to the internet. After deployment, verification of the application's status will be conducted using kubectl commands to ensure that it is running correctly.

2. Step-by-Step Explanation

Prerequisites:

1. AWS Account with proper permissions.
2. AWS CLI and kubectl installed in AWS CloudShell (default in CloudShell).
3. Terraform set up on your local machine or CloudShell.

Step 1: Setup AWS CloudShell



Step 2: Install Terraform in AWS CloudShell (if not already installed)

Download Terraform:

curl -O

https://releases.hashicorp.com/terraform/1.5.6/terraform_1.5.6_linux_amd64.zip

Unzip the file:

unzip terraform_1.5.6_linux_amd64.zip

Move Terraform binary to /usr/local/bin/:

sudo mv terraform /usr/local/bin/

Verify Terraform installation:

terraform -v

```
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ terraform -v
Terraform v1.9.8
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v5.72.1
+ provider registry.terraform.io/hashicorp/cloudinit v2.3.5
+ provider registry.terraform.io/hashicorp/kubernetes v2.33.0
+ provider registry.terraform.io/hashicorp/time v0.12.1
+ provider registry.terraform.io/hashicorp/tls v4.0.6
[cloudshell-user@ip-10-132-80-173 eks-terraform]$
```

Step 3: Configure AWS CloudShell Environment

Set up the AWS CLI to ensure you are using the correct region and account:

aws configure

Enter your AWS Access Key, Secret Key, Default region (e.g., us-east-1), and Default output format (json).

Verify kubectl is installed:

kubectl version --client

```
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ kubectl version --client
Client Version: v1.30.2-eks-1552ad0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
[cloudshell-user@ip-10-132-80-173 eks-terraform]$
```

Step 4: Write a Terraform Script to Create an EKS Cluster

Create a working directory:

mkdir eks-terraform cd eks-terraform

Create a file main.tf for the EKS cluster and its resources

nano main.tf

main.tf

Configure the AWS Provider

provider "aws" {

region = "ap-south-1" # Change to your desired region

}

Use existing subnets in the default VPC

data "aws_vpc" "default" {

default = true

}

data "aws_subnets" "default_subnets" {

filter {

name = "vpc-id"

values = [data.aws_vpc.default.id]

}

}

Select the first two subnets

locals {

subnet_ids = slice(data.aws_subnets.default_subnets.ids, 0, 2)

```
}
```

```
module "eks" {  
  source      = "terraform-aws-modules/eks/aws"  
  version     = "~> 19.0" # Specify the version you want to use  
  cluster_name = "my-cluster"  
  cluster_version = "1.25"  
  subnet_ids   = local.subnet_ids  
  vpc_id       = data.aws_vpc.default.id  
  
  eks_managed_node_groups = {  
    eks_nodes = {  
      desired_size = 2  
      max_size     = 3  
      min_size     = 1  
      instance_types = ["t3.medium"]  
    }  
  }  
}
```

```
output "cluster_endpoint" {  
  description = "Endpoint for EKS control plane"  
  value       = module.eks.cluster_endpoint  
}
```

```
output "cluster_security_group_id" {  
    description = "Security group ids attached to the cluster control plane"  
    value      = module.eks.cluster_security_group_id  
}
```

Additional useful outputs

```
output "cluster_name" {  
    description = "The name of the EKS cluster"  
    value      = module.eks.cluster_name  
}
```

```
output "cluster_arn" {  
    description = "The Amazon Resource Name (ARN) of the cluster"  
    value      = module.eks.cluster_arn  
}
```

Initialize Terraform:

terraform init

Plan and apply the changes to create the EKS cluster:

terraform apply

Review the plan output, then confirm the changes by typing yes.

This will take a few minutes to provision the EKS cluster.

```
Apply complete! Resources: 30 added, 0 changed, 0 destroyed.
```

Outputs:

```
cluster_arn = "arn:aws:eks:ap-south-1:108782066834:cluster/my-cluster"
cluster_endpoint = "https://4E89B0F21B68FB0207030395C0A9E351.gr7.ap-south-1.eks.amazonaws.com"
cluster_name = "my-cluster"
cluster_security_group_id = "sg-0b0ad9fe7a76aea7a"
[cloudshell-user@ip-10-132-80-173 eks-terraform]$
```

```
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ aws eks describe-cluster --name
my-cluster --query cluster.resourcesVpcConfig
```

```
{
  "subnetIds": [
    "subnet-022ceb74e62203ccd",
    "subnet-02d06c8568ef0dd6f"
  ],
  "securityGroupIds": [
    "sg-0b0ad9fe7a76aea7a"
  ],
  "clusterSecurityGroupId": "sg-00c2ae21910dc2962",
  "vpcId": "vpc-056a6e4674f2f91fd",
  "endpointPublicAccess": false,
  "endpointPrivateAccess": true,
  "publicAccessCidrs": [
    "0.0.0.0/0"
  ]
}
```

This is the cluster configuration will look like

Step 5: Configure kubectl for EKS Cluster Access

Once the EKS cluster is created, configure kubectl to interact with it:

Run the following command to update the kubeconfig file with your new cluster:

```
aws eks --region <region> update-kubeconfig --name <cluster_name>
```

In this case cluster name-my-cluster

Region - ap-south-1

Step 6: Deploy a Sample Python Flask App on Kubernetes

Create a deployment for your Flask app.

Create a file flask-app.yaml:

Nano flask-app.yaml

flask-app.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask
  template:
    metadata:
      labels:
        app: flask
    spec:
      containers:
        - name: flask-container
          image: tiangolo/uwsgi-nginx-flask:python3.8
          ports:
            - containerPort: 80
```

Apply the deployment:

```
kubectl apply -f flask-app.yaml
```

Expose the deployment using a LoadBalancer service to access it publicly:

```
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  type: LoadBalancer
  selector:
```

app: flask

ports:

- protocol: TCP

port: 80

targetPort: 80

Apply the service configuration:

Run the following commands:

kubectl apply -f flask-service.yaml

Get the external IP of the service:

kubectl get svc flask-service

Wait for the external IP to be assigned and navigate to the IP in your browser.

```
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ kubectl apply -f flask-service.yaml
service/flask-service unchanged
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ kubectl get svc flask-service
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP      PORT(S)          AGE
flask-service LoadBalancer  10.100.62.214  a1422385fc573493cba15b46dc862164-551218818.ap-south-1.elb.amazonaws.com  80:31485/TCP    91m
[cloudshell-user@ip-10-132-80-173 eks-terraform]$
```

Step 7: Verify Deployment

Once the Flask app is deployed and the LoadBalancer has an external IP, you can access the application in a web browser using the external IP address.

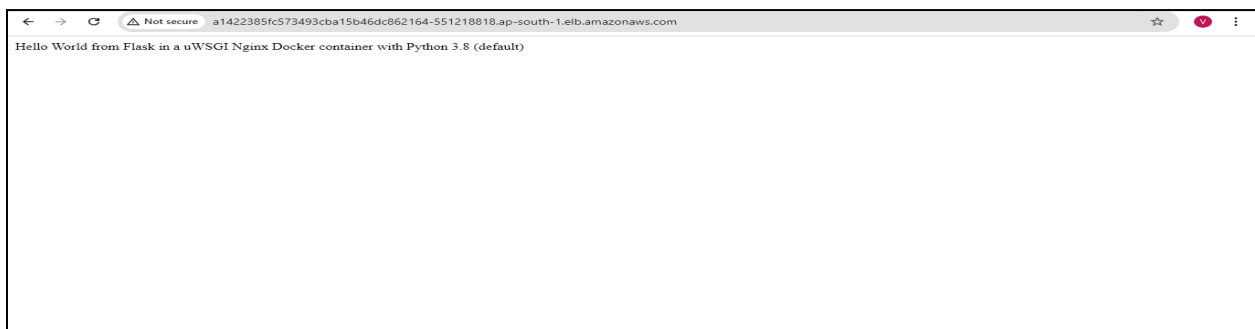
Run the following to check the status of your pods and services:

kubectl get pods

kubectl get svc

```
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
flask-app-6fbdcc5bc-t9czq  1/1     Running   0          4m38s
flask-app-6fbdcc5bc-w79vm  1/1     Running   0          4m38s
[cloudshell-user@ip-10-132-80-173 eks-terraform]$ kubectl get svc
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP      PORT(S)          AGE
flask-service LoadBalancer  10.100.62.214  a1422385fc573493cba15b46dc862164-551218818.ap-south-1.elb.amazonaws.com  80:31485/TCP    18s
kubernetes   ClusterIP      10.100.0.1     <none>           443/TCP          90m
[cloudshell-user@ip-10-132-80-173 eks-terraform]$
```

Step 8: Open browser and check if the flask app is running or not



3. Demonstration Preparation

Key Points to Focus on During the Demonstration:

1. **Clear Objective:** Start by explaining the goal of the practical or project clearly. In this case, it's deploying and managing a Kubernetes cluster with Terraform, and deploying a Flask app on AWS.
2. **Tools and Technologies:** Highlight the tools being used—AWS (CloudShell, EKS), Terraform for infrastructure provisioning, Kubernetes for managing containers, and Flask for the application.
3. **Step-by-Step Process:** Walk through the main steps:
4. Writing a Terraform script to provision the EKS cluster.
5. Configuring kubectl for cluster access.
6. Deploying a Flask application and exposing it with a LoadBalancer service.
7. **Key Challenges:** Mention any potential challenges, such as troubleshooting failed pod deployments or external IP allocation issues.
8. **Verification:** Explain how you verified successful deployment by checking pod statuses and accessing the Flask app using the external IP.
9. **Cleanup:** Discuss the importance of cleaning up resources after testing using terraform destroy.
10. **Why It Matters:** Connect the demonstration to real-world use cases, such as scalable application deployment on cloud platforms, and emphasize how infrastructure as code (Terraform) simplifies cloud resource management.
11. **Importance of Practice:**
12. **Familiarity with Tools:** Practicing the demonstration ensures you're familiar with AWS CloudShell, Terraform commands, and kubectl commands, which is critical for a smooth demo.
13. **Anticipating Issues:** During practice, you may encounter issues such as configuration errors or resource limits on AWS Free Tier, which will prepare you to resolve them quickly during the actual demo.
14. **Confidence:** The more you practice, the more confident you'll be in explaining complex concepts in simpler terms, which is crucial for a successful presentation.
15. **Time Management:** Practice helps you streamline the demo to fit the allotted time and focus on the most important aspects without unnecessary delays.
16. **Rehearse Troubleshooting:** Practicing different scenarios, such as a failed deployment, lets you showcase problem-solving skills during the demo.
17. **Flow of the Presentation:** Ensure the presentation flows logically—from infrastructure provisioning to app deployment—so that your audience can easily follow along.

4. Certification Requirement

Deadline:

As part of the case study project, it is important to meet the certification deadline set for October 21st. Completing the tasks and ensuring all requirements are fulfilled before this date is crucial for the successful submission of this practical.

The deadline serves as a key milestone, emphasizing the need to complete the infrastructure deployment, app configuration, and testing phases promptly. By adhering to this schedule, you demonstrate strong project management skills and a commitment to achieving your objectives within the given time frame.

Importance of Certification:

Obtaining certification in cloud technologies, Kubernetes, and Terraform plays a significant role in validating the skills acquired through this case study. Certification demonstrates your ability to:

Provision and manage Kubernetes clusters using Terraform as Infrastructure as Code (IaC), which is a critical skill in modern cloud infrastructure management.

Deploy and manage containerized applications efficiently using Kubernetes, an essential requirement in DevOps and cloud-native environments.

Integrate AWS services with Terraform and Kubernetes, showcasing proficiency in managing cloud infrastructure on leading platforms.

Achieving certification also opens up opportunities for career advancement by providing formal recognition of your expertise. For this project, certification is not just a requirement but a formal endorsement of your proficiency in cloud infrastructure management.

By completing this case study within the given timeline and earning certification, you'll not only meet the academic requirements but also gain valuable credentials that are highly sought after in the industry.